

Calling system calls

As we have already discussed, system calls or, more precisely, system call handlers are a part of kernel code, which are executed in the kernel mode (high privilege mode 0). Therefore, it is not possible for user space applications or glibc wrapper to execute kernel code directly. To change the mode from user to kernel space, glibc wrapper function must somehow signal system call handler execution to the kernel.

In the older version of x86 processors, Linux kernel and glibc, system call wrappers used to trigger a software interrupt (exception), and the system would switch to kernel mode and execute the exception handler. The exception handler, in this case, is the system call handler. The defined software interrupt on x86 is interrupt number 128, which is incurred via the `int $0x80` instruction. It triggers a switch to kernel mode and the execution of exception vector 128, which is the system call handler.

However, the `int` assembly language instruction is inherently slow because it performs several consistency and security checks. Nowadays, modern x86 processors provide a faster way of system call through the execution of `syscall` (on x86-64 architecture) instruction. The newer version of glibc system call wrapper executes `syscall` instruction for switching the mode from user space to kernel space and execution of system call handler.

We now understand that glibc system call wrapper sets the argument into the specified registers (as directed by kernel) and executes `syscall` instruction to hand over the work to the kernel space. With this knowledge, let's browse the glibc wrapper of `open(2)` system call to check how things are actually implemented in C library. Here's the code snippet that gives the details of implementation. Check the code comment marked in red.

[sysdeps/unix/sysv/linux/generic/open.c](#)

```
int
__libc_open (const char *file, int oflag, ...)
{
    int mode = 0;

    if (__OPEN_NEEDS_MODE (oflag))
    {
        va_list arg;
        va_start (arg, oflag);
        mode = va_arg (arg, int);
        va_end (arg);
    }

    if (SINGLE_THREAD_P)
        return INLINE_SYSCALL (openat, 4, AT_FDCWD, file, oflag, mode);

    int oldtype = LIBC_CANCEL_ASYNC ();
```

```
    /*
     * internally glibc is invoking openat(2) system call
     * INLINE_SYSCALL macro is used for invoking the system call
     */

    int result = INLINE_SYSCALL (openat, 4, AT_FDCWD, file,
                                oflag, mode);

    LIBC_CANCEL_RESET (oldtype);

    return result;
}
libc_hidden_def (__libc_open)

weak_alias (__libc_open, __open)
libc_hidden_weak (__open)
weak_alias (__libc_open, open)

sysdeps/unix/sysv/linux/x86\_64/sysdep.h

# undef INLINE_SYSCALL
# define INLINE_SYSCALL(name, nr, args...) \
    ({

/* Further INTERNAL_SYSCALL macro is being invoked*/

        unsigned long int resultvar = INTERNAL_SYSCALL (name, ,
nr, args); \
        if (__glibc_unlikely (INTERNAL_SYSCALL_ERROR_P
(resultvar, ))) \
        {
            \
            __set_errno (INTERNAL_SYSCALL_ERRNO (resultvar, )); \
            resultvar = (unsigned long int) -1; \
        }
        (long int) resultvar; })

#define INTERNAL_SYSCALL(name, err, nr, args...) \
    internal_syscall##nr (SYS_ify (name), err, args)

#undef SYS_ify
#define SYS_ify(syscall_name)    __NR_##syscall_name

/* SYS_ify() macro will convert the name into system call
number NR_syscall_name.

* These system call number are defined
* /usr/include/x86_64linuxgnu/asm/unistd_64.h
* #define __NR_read    0
* #define __NR_write   1
* #define __NR_open    2
* #define __NR_close   3
* #define __NR_stat    4
```